

Распараллеливание циклов в OpenMP

Как показывает практика, наибольший ресурс параллелизма в программах сосредоточен в циклах. Поэтому наиболее распространенным способом распараллеливания является то или иное распределение итераций циклов. Если между итерациями некоторого цикла нет информационных зависимостей, то их можно каким-либо способом раздать разным нитям для одновременного исполнения.

Если в параллельной области встречается цикл, то он будет выполнен всеми нитями текущей группы, каждая нить выполнит все итерации данного цикла. Для распределения итераций цикла между различными нитями необходимо добавить директиву `for`:

```
#pragma omp for [ ОПЦИЯ . . , ОПЦИЯ . . ]
```

Эта директива относится к идущему следом за данной директивой блоку, включающему оператор `for`.

Опции для директивы `for`:

Опция `private` (список переменных) — список переменных локального хранения для каждой нити.

Опция `firstprivate` (список переменных) — получают свои значения из последовательной части программы.

Опция `lastprivate` (список переменных) — сохраняют свои значения при выходе из параллельных нитей.

Опция `reduction` (оператор : список переменных) — выполняет действия с указанными переменными.

Опция `schedule` (тип [, размер итерации]) — задает каким образом итерации цикла распределятся между нитями.

Опция `collapse(uint)` — указывает, что некоторое число последовательных тесновложенных циклов ассоциируется данной директивой. Для циклов образуется общее пространство итераций, которое делится между нитями. Если опция `collapse` не задана, то директива `for` относится только к одному непосредственно следующему за ней циклу.

Опция `ordered` — указывает на то, что цикле могут встречаться директивы `ordered`. В таком случае определяется блок в теле цикла, который должен будет выполняться в том порядке, в котором итерации идут в последовательном цикле.

Опция `nowait` — отменяет в конце параллельного цикла барьерную синхронизацию.

На вид циклов, которые можно сделать параллельными, накладываются ограничения. Как правило это должен быть цикл `for`. Предполагается, что не должно быть зависимостей в цикле, программа не должна зависеть от того, какая именно нить какую итерацию цикла выполняет. Нельзя использовать побочные выходы из параллельного цикла. Размер блока итераций, который указывается в блоке `schedule`, не должен изменяться в рамках цикла. Формат параллельного цикла в упрощенной форме может выглядеть таким образом:

```
for ( [целочисленный тип] i = инвариант цикла;  
i (>, <, =, <=, >=) инвариант цикла; i (+, -) =  
инвариант цикла )
```

Пример использования директивы `for` в уже объявленном параллельном блоке:

```
#pragma omp parallel
{
    int id = omp_get_thread_num();
    #pragma omp for
    for(int i = 0; i < 10; i++) {
        printf( "Нить: %d, Итерация: %d\n" , id, i );
    }
}
```

Итерации цикла, помеченного директивой `for`, будут распределены между теми нитями, которые порождены директивой `parallel`.

Различные способы распределения итераций позволяют добиваться максимально равномерной загрузки нитей, между которыми распределяются итерации цикла.

Статический способ распределения итераций позволяет уже в момент написания программы точно определить, какой нити достанутся какие итерации. Однако он не учитывает текущей загруженности процессоров, соотношения времён выполнения различных итераций и некоторых других факторов.

Динамическое планирование помогает в проблемах с балансировкой нагрузки, в задачах, где каждая итерация цикла может занимать разные промежутки времени, а статическое распределение итераций скорее всего создаст дисбаланс работы между различными потоками. Дисбаланс работы приводит к расходованию процессорного времени, поскольку потоки, которые завершают работу раньше, просто ждут окончания остальных потоков. Динамическое планирование решает отложенным способом разделение нагрузки на вычислительную систему, но в тоже время может быть более ресурсозатратным, чем статическая модель.

В опции `schedule` параметр `type` задаёт несколько именованных типов распределения итерации:

`static` — блочно-циклическое распределение итераций цикла. Размер блока — `chunk`. Первый блок из `chunk` итераций выполняет нулевая нить, второй блок — следующая и так далее до последней нити, затем распределение начинается снова с нулевой нити. Если значение `chunk` не указано, то всё множество итераций делится на части примерно одинакового размера и все нити получают примерно одинаковую нагрузку, либо нескольким последним нитям достанется чуть меньше, в том случае если число итераций нацело не делится.

`dynamic` — динамическое распределение итераций цикла с фиксированным размером блока. Для начала каждая нить получает `chunk` итераций (по умолчанию `chunk = 1`). Как только одна из нитей заканчивает выполнение своей порции итераций, то ей отдаются новые порцию из `chunk` итераций. Последняя порция может содержать меньше итераций, чем все остальные.

`guided` — динамическое распределение итераций цикла, при котором размер порций уменьшается. Параметр `chunk` определяет начальный размер порции итераций. По сути для `guided` не критично указания размера порции явно, поскольку OpenMP автоматически регулирует размер порции в зависимости от характера выполнения цикла.

`auto` — способ распределения итераций выбирается компилятором и/или системой выполнения. Параметр `chunk` при этом не задается и выбирается автоматически.

`runtime` — способ распределения итераций выбирается во время работы по значению переменной среды `OMP_SCHEDULE`. Параметр `chunk` при этом не задается.

Пример программы, использующий статическое и динамическое планирование:

```
#pragma omp parallel private(i)
{
    #pragma omp for schedule(static, 2)
    for(int i = 0 ; i < 10 ; ++i){
        printf( "Нить %d выполнила итерацию %d\n",
omp_get_thread_num(), i);
    }

    #pragma omp parallel for schedule(dynamic)
    for (int i = 0; i < 10; ++i) {
        printf( "Нить %d выполнила итерацию %d\n",
omp_get_thread_num(), i);
    }
}
```

Стратегия распределение нагрузки при распараллеливании циклов может быть задано с помощью переменной `OMP_SCHEDULE`, задать значение переменной `OMP_SCHEDULE` можно с помощью команды:

```
export OMP_SCHEDULE="dynamic,1"
```

Изменить значение переменной `OMP_SCHEDULE` из программы можно с помощью вызова функции `omp_set_schedule()`. Процедура имеет вид

```
void omp_set_schedule(omp_sched_t type, int chunk)
```

Допустимые значения констант описаны в файле `omp.h`. Существуют, как минимум, следующие варианты:

```
typedef enum omp_sched_t {  
    omp_sched_static=1,  
    omp_sched_dynamic=2,  
    omp_sched_guided=3,  
    omp_sched_auto=4  
}
```

При помощи процедуры `omp_get_schedule()` есть возможность проверить текущее значение `OMP_SCHEDULE`.

```
void omp_get_schedule( omp_sched_t * type,  
int* chunk );
```

При распараллеливании цикла нужно убедиться, что итерации данного цикла не имеют информационных зависимостей. В таком случае итерации можно будет выполнить в любом порядке, в том числе параллельно.

Компилятор не проверяет информационную зависимость в циклах, возможны логические ошибки в результатах программы если этого не учесть.

Исполнение параллельных секций OpenMP

Директива `sections` определяет набор независимых секций кода, каждая из которых выполняется своей нитью.

```
#pragma omp sections [опция...]
```

Список опций:

`private(список)`

`firstprivate(список)`

`lastprivate(список)` – переменным присваивается

результат из последней секции

`reduction(оператор:список)`

`nowait`

Директива `section` задаёт участок кода внутри секции `sections` для выполнения одной нитью.

```
#pragma omp section
```

Перед первым участком кода в блоке `sections` директива `section` не обязательна. Какие нити будут задействованы для выполнения какой секции, не специфицируется. Если количество нитей больше количества секций, то часть нитей для выполнения данного блока секций не будет задействована. Если количество нитей меньше количества секций, то некоторым (или всем) нитям достанется более одной секции.

Пример, демонстрирующий параллельное исполнение секций:

```
int n;
#pragma omp parallel private(n)
{
    n = omp_get_thread_num();
    #pragma omp sections
    {
        #pragma omp section
        printf( "Первая секция, процесс %d\n", n );
        #pragma omp section
        printf( "Вторая секция, процесс %d\n", n );
        #pragma omp section
        printf( "Третья секция, процесс %d\n", n );
    }
}
```



```
int n=0;
#pragma omp parallel
{
    #pragma omp sections lastprivate(n)
    {
        #pragma omp section
            n=1;
        #pragma omp section
            n=2;
    }
    printf( "Значение n в контексте нити %d: %d\n",
            omp_get_thread_num( ), n );
}
printf( "Значение n в конце: %d\n", n );
```

Директива `task`

Для реализации подхода декомпозиции по задачам в OpenMP предусмотрена специальная директива `task`.

```
#pragma omp task [опция ...]
```

Текущая нить выделяет в качестве задачи ассоциированный с директивой блок операторов.

Задача может выполняться немедленно после создания или быть отложенной на неопределённое время и выполняться по частям. Размер таких частей, а также порядок выполнения частей разных отложенных задач определяется реализацией.

`if(условие)` — если значение выражения отлично от нуля, то новая задача будет порождена, иначе код структурного блока будет немедленно выполнен текущей нитью;

`untied` — если выполнение задачи прервано, то её может продолжить выполнять любая нить в параллельной области. Если опция не указана, то продолжить выполнение отложенной задачи может только нить, которая изначально взяла её на выполнение;

`final(выражение)` — если значение выражения отлично от нуля, то новая задача будет порождена как конечная, т. е. все вложенные задачи также будут конечные и должны безотлагательно выполняться, причем той же самой нитью, что и родительская. При истинности выражения в `final` опция `untied`, если она присутствует в этой же директиве, игнорируется. Использование опции полезно для ограничения порождения новых нитей при параллельной рекурсии, поскольку глубокая рекурсия может исчерпать аппаратные и/или программные ограничения на порождение новых нитей;

`mergeable` — указывает, что если порожденная задача «погружена» в родительскую, то она будет использовать одни и те же данные;

`default (shared|none)`

`private (список)`

`firstprivate (список)`

`shared (список)`

Для управления задачами в OpenMP определены две директивы:

`#pragma omp taskyield` — эта директива сообщает, что задачу можно в этой точке прервать для выполнения других задач. Если задача порождена с опцией `untied`, то её может затем продолжить выполнять любая нить группы.

Для гарантированного завершения в точке вызова всех запущенных задач используется директива `taskwait`.

`#pragma omp taskwait` — нить, выполнившая данную директиву, приостанавливается до тех пор, пока не будут завершены все ранее запущенные данной нитью независимые задачи.

Пример использования директивы task:

```
typedef struct task_argument_s{
    unsigned task_id;
    unsigned task_wait;
} task_argument_t;

static void processElement ( task_argument_t *task_arg )
{
    #pragma omp critical
    printf ( "Поток %d начал исполнение задачи %u \n", omp_get_thread_num(), task_arg->task_id );
    // Исполнение длительной задачи
    usleep ( task_arg->task_wait * 1000 );
    #pragma omp critical
    printf ( "Поток %d завершил исполнение задачи %u \n", omp_get_thread_num(), task_arg->task_id );
}

int main (void){
    task_argument_t task_arguments[] = {{0, 100}, {1, 1000}, {2, 20}};

    #pragma omp parallel // Создание параллельной секции
    #pragma omp single // Только один поток будет добавлять задачу
    {
        printf("Доступное количество нитей: %d \n», omp_get_num_threads());
        for ( int i = 0 ; i < sizeof(task_arguments)/sizeof(task_argument_t) ; ++i ){
            task_argument_t *argument = task_arguments + i;
            #pragma omp task firstprivate (argument)
            processElement ( argument );
        }
        #pragma omp taskwait
        printf( "Все задачи выполнены!\n" );
    }
    return 0;
}
```

Пример taskyield:

```
void sth_useful ( void );
void sth_critical ( void );
void foo ( omp_lock_t * lock, int n )
{
    int i;
    for ( i = 0; i < n; i++ )
        #pragma omp task
        {
            sth_useful();
            while ( !omp_test_lock(lock) ) {
                #pragma omp taskyield
            }
            sth_critical();
            omp_unset_lock(lock);
        }
}
```

Пример taskwait:

```
int fibonacci(int n) {
    int i, j;
    if (n<2) {
        return n;
    } else {
        #pragma omp task shared(i)
        i=fibonacci (n-1);
        #pragma omp task shared(j)
        j=fibonacci (n-2);
        #pragma omp taskwait
        return i+j;
    }
}

int main () {
    int res;
    #pragma omp parallel
    {
        #pragma omp single
        {
            int n = 40;
            #pragma omp task shared(res)
            res = fibonacci(n);
        }
    }
    printf ("Finonacci number =  %d\n", res);
}
```


Пример `taskwait`:

```
#pragma omp task {}           // Task1
#pragma omp task               // Task2
{
    #pragma omp task {}       // Task3
}
#pragma omp task {}           // Task4
```

`#pragma omp taskwait` // в данной точке завершатся
Task1, Task2, Task4

Задачи можно группировать с помощью опции `taskgroup`:

```
#pragma omp task {}           // Task1
#pragma omp taskgroup
{
    #pragma omp task           // Task2
    {
        #pragma omp task {}    // Task3
    }
    #pragma omp task {}        // Task4
} // в данной точке завершатся
Task2, Task3, Task4
```

Директива `taskloop`

Начиная со стандарта OpenMP 4.5 появилась возможность не просто отдавать на выполнение всем потокам итераций по частям, а создавать для них задачи (`task` 'и), которые будут распределяться по потокам. Реализуется это с помощью конструкции `taskloop`:

```
#pragma omp taskloop
```

Пример использования директивы `taskloop`:

```
#define SIZE 10
```

```
int main() {
```

```
    double timeStart, timeEnd;  
    timeStart = omp_get_wtime();
```

```
    #pragma omp parallel
```

```
    #pragma omp single
```

```
    #pragma omp taskloop
```

```
    for( int i = 0 ; i < SIZE ; ++i ) {  
        printf( "ThreadId: %d, taskid: %d\n",
```

```
omp_get_thread_num(), i);  
    }
```

```
    timeEnd = omp_get_wtime();
```

```
    printf( "Время выполнения %lf\n", timeEnd - timeStart);
```

```
}
```